⚙️ Clase 5 – Funciones y Modularización (Versión Completa)

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano

📓 Contenidos

- Repaso de punteros
- Definición de funciones y procedimientos
- Variables locales y globales
- Parámetros por valor y por referencia
- Recursividad
- Modularización avanzada (`typedef`, `struct`, `#include`, `.h`, `.c`)
- Arquitectura en capas de software

🎯 Objetivos

OnlineGDB es un entorno de desarrollo en la nube (IDE) que permite editar, compilar y ejecutar programas en **C/C++** fácilmente:
🔗 https://www.onlinegdb.com/

📍 Repaso: Punteros

Punteros (repaso)

¿ Qué es un puntero ?

- Un puntero es una variable especial que guarda direcciones de memoria
- Se dice que “apunta a” dirección de memoria, que contiene un valor con un tipo específico de dato, definido previamente.

¿ Cómo se define ?

- tipo *nombrePuntero;
- Se utilizan los operadores de referencia (&) y desreferencia (*)
- Operador de referencia (&): obtiene la direccion de memoria de una variable
- Operador de desreferencia (*): acceder al valor que hay en esa dirección.

```
#include <stdio.h>

int main() {
```

```
    int x = 10;
    int *ptr = &x;  // ptr apunta a la dirección de x

    printf("Valor de x: %d\n", x);
    printf("Dirección de x: %p\n", &x);
    printf("Valor apuntado por ptr: %d\n", *ptr);
    return 0;
}
```

Representación en memoria:

```
+-----------+-----------+-------+
| Variable  | Dirección | Valor |
+-----------+-----------+-------+
| x         | 1000      | 10    |
| ptr       | 1004      | 1000  |
+-----------+-----------+-------+
```

¿Por qué modularizar?

¿Por qué modularizar?

- Problema: un programa lineal crece y se vuelve inmanejable.
- Solución: divide y vencerás → separar en subprogramas.
- En C: todo son funciones (aunque algunas funcionan como procedimientos).
- Un solo bloque → difícil de mantener
- Dividir en funciones → claro, reutilizable, escalable

```
┌──────────────────────────────────────────────┐
│ Programa grande y lineal                     │
│ └→ difícil de mantener y entender            │
│ SOLUCIÓN: dividirlo en funciones pequeñas    │
│ └→ más claridad, reutilización y escalabilidad │
└──────────────────────────────────────────────┘
```

```
#include <stdio.h>

// Prototipos
int sumar(int a, int b);
int restar(int a, int b);
void mostrarResultado(int valor);
void duplicarPorReferencia(int *valor);

int main() {
    int x = 5, y = 3;
```

```
    int resultado;

    resultado = sumar(x, y);
    mostrarResultado(resultado);

    resultado = restar(x, y);
    mostrarResultado(resultado);

    duplicarPorReferencia(&x);
    mostrarResultado(x);

    return 0;
}

// Definiciones
int sumar(int a, int b) {
    return a + b;
}

int restar(int a, int b) {
    return a - b;
}

void mostrarResultado(int valor) {
    printf("Resultado: %d\n", valor);
}

void duplicarPorReferencia(int *valor) {
    *valor = *valor * 2;
}
```

🔧 Funciones en C

Concepto

- Una función devuelve un valor con return.
- Representa un cálculo o transformación.
- Facilita reutilización y claridad.

Estructura de declaración de función

tipo nombre_funcion(tipo1 param1, tipo2 param2); En la definición de la funcion, hay que devolver el valor con return int sumar(int a, int b) { return a + b; }

Ejemplo simple

```
#include <stdio.h>
```

```c
int sumar(int a, int b) {
    return a + b;
}

int main() {
    int resultado = sumar(5, 7);
    printf("La suma es: %d\n", resultado);
    return 0;
}
```

Estructura general:

```c
tipo nombre_funcion(tipo1 param1, tipo2 param2);
```

Procedimientos (`void`)

Las funciones que **no devuelven valor** se declaran con `void`:

Concepto

- Un procedimiento no devuelve valor (en C también es una función)
- Su objetivo es realizar una acción, no un cálculo.
- Se declara con void

Estructura de declaración de procedimiento

void nombre_proc(tipo1 param1, tipo2 param2); Ejemplo: void imprimirSuma(int a, int b);

```c
#include <stdio.h>

void imprimirSuma(int a, int b) {
    printf("La suma es: %d\n", a + b);
}

int main() {
    imprimirSuma(4, 8);
    return 0;
}
```

Tipos `void` y punteros genéricos

Concepto

- void es un tipo de dato especial que significa “sin valor” o “nada”.

- Se utiliza en dos contextos principales:
  - Funciones sin valor de retorno (procedimientos)
  - Funciones sin parámetros
  - Punteros genéricos (void *)
- Un puntero generico, puede apuntar cualquier tipo de dato

```c
void *punteroGenerico;
int x = 10;
punteroGenerico = &x;
printf("%d", *(int*)punteroGenerico); // Conversión necesaria
```

```c
void mostrarMensaje() {
    printf("Hola!\n");

}
int generarNumero(void) {
    return 42;
}
void *ptr;  // puntero genérico
```

`void *` puede apuntar a cualquier tipo de dato.

Alcance de variables

Variables locales y globales

- Variables locales
  - Se declaran dentro de una función.
  - Solo son visibles en esa función.
  - Su tiempo de vida dura mientras la función se ejecuta.
- Variables globales
  - Se declaran fuera de las funciones (antes de main).
  - Son visibles y utilizables en todo el programa.
  - Se recomienda usarlas con cuidado para evitar errores y confusión.

```c
#include <stdio.h> // inclusion de libreria

// declaracion de constante
#define PI 3.1415

// declaracion de variable global
int contadorGlobal = 0;

// prototipo de funcion
```

```c
int sumar(int a, int b);

int main(void) {
    // Declaracion de variables locales
    int x, y, resultado;

    contadorGlobal = 1; //modifico la variable global

    return 0; // devuelve 0 al sistema operativo -> ejecución correcta
}

// implementacion de funcion
int sumar(int a, int b) {
    return a + b;
}
```

Parámetros: Por valor y por referencia

- Por copia (valor)
  - Se pasa una copia de la variable. Lo que cambie dentro de la función no afecta al original.
- Por referencia (dirección)
  - Se pasa un puntero, la función puede modificar la variable original

Modo	Qué pasa	Efecto	Uso típico
Por valor	Se pasa una copia	No modifica el original	Operaciones de solo lectura
Por referencia	Se pasa la dirección	Modifica el original	Cuando necesitamos devolver más de un resultado

Ejemplo por valor

```c
void duplicar(int n) {
    n = n * 2;
}

int main() {
    int x = 5;
    duplicar(x);
    printf("%d\n", x); // 5 (no cambió)
}
```

Ejemplo por referencia

```c
void duplicar(int *n) {
    *n = *n * 2;
}

int main() {
    int x = 5;
    duplicar(&x);
    printf("%d\n", x); // 10 (modificado)
}
```

📊 Pasaje de vectores como parámetros

Un vector se pasa como **puntero al primer elemento**.

> Ejemplo: pasar un vector como parámetro y calcular el promedio

En C, cuando usamos el nombre de un vector (por ejemplo v), este se interpreta como la dirección de memoria donde comienza el arreglo, es decir, la dirección de su primer elemento

El nombre de un vector entonces es equivalente a un puntero al primer elemento.

- v →dirección de v[0]
- *v →contenido de v[0]

(página 21 - Manual de referencia)

```c
#include <stdio.h>
#define N 5 // tamaño del vector

// Función que recibe un vector por referencia y devuelve el promedio
float promedio(int v[], int n) {
    int suma = 0;
    for (int i = 0; i < n; i++) {
        suma += v[i];
    }
    return (float)suma / n;
}

// Procedimiento para mostrar el vector
void mostrar(int v[], int n) {
    int i;
    printf("Vector:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("\n");
}

int main() {
    int vec[N] = {10, 20, 30, 40, 50}; // vector de ejemplo
```

```
    mostrar(vec, N); // paso el vector a un procedimiento
    printf("El promedio es: %.2f\n", promedio(vec, N));
    return 0;
}
```

🔁 Funciones recursivas

¿Qué es la recursividad?

- Una función se llama a sí misma dentro de su definición.
- Se utiliza para problemas que pueden descomponerse en subproblemas más pequeños.
- Siempre debe haber una condición de corte para evitar bucles infinitos.

```c
#include <stdio.h>

int factorial(int n) {
    if (n == 0) return 1; // Caso base
    return n * factorial(n - 1); // Llamada recursiva
}

int main() {
    printf("Factorial de 5 = %d\n", factorial(5));
    return 0;
}
```

🤔 **Visualización:**

```
factorial(5)
→ 5 * factorial(4)
→ 5 * 4 * factorial(3)
→ 5 * 4 * 3 * factorial(2)
→ 5 * 4 * 3 * 2 * factorial(1)
→ 5 * 4 * 3 * 2 * 1 = 120
```

🧱 Modularización avanzada

¿Por qué modularizar?

- Problema: un programa lineal crece y se vuelve inmanejable.
- Solución: divide y vencerás → separar en subprogramas.
- En C: todo son funciones (aunque algunas funcionan como procedimientos).
- Un solo bloque → difícil de mantener

- Dividir en funciones → claro, reutilizable, escalable

```
#include <stdio.h>

// Prototipos
int sumar(int a, int b);
int restar(int a, int b);

int main() {
    int x = 5, y = 3;
    int resultado;

    resultado = sumar(x, y);
    mostrarResultado(resultado);

    resultado = restar(x, y);
    mostrarResultado(resultado);

    duplicarPorReferencia(&x);
    mostrarResultado(x);

    return 0;
}

// Definiciones
int sumar(int a, int b) {
    // return a + b;
}

int restar(int a, int b) {
    // return a - b;
}
```

`typedef` — alias para tipos

- typedef permite darle un alias a un tipo existente → más claro, más legible, más portable.
- Facilita la lectura del código.
- Evita repetir tipos largos.
- Permite adaptar fácilmente el programa si cambia el microcontrolador o compilador.
- Sintaxis general typedef tipo_existente nuevo_nombre;

```
typedef unsigned int uint;
typedef float temperatura;

uint edad = 25;
temperatura sensor = 23.7;
```

```c
// Tipos de medidas y unidades
typedef float temperatura_t;
typedef double distancia_t;     // en metros
typedef float peso_t;           // en kilogramos
typedef long tiempo_t;          // en segundos

int main() {
    temperatura_t temp_actual = 20.1;
    distancia_t distancia = 10.2;
    peso_t peso = 10;

    printf("Temperatura: %.1f°C\n", temp_actual);
    printf("Distancia de A a B: %.1f\n", distancia);
    printf("Peso: %.1f\n", peso);

    return 0;
}
```

`struct` — agrupar datos relacionados

- struct permite empaquetar variables distintas en un solo bloque lógico.
- Representa un objeto del mundo real (ej: sensor, alumno, producto).
- Se puede combinar con typedef para simplificar el uso

```c
typedef float temperatura_t;
typedef float humedad_t;
typedef int calidad_aire_t;
typedef long timestamp_t;

typedef struct {
    temperatura_t temp;
    humedad_t humedad;
    calidad_aire_t calidad_aire;
    timestamp_t timestamp;
    lectura_sensor_t;
} lectura_sensor_t;

// Procedimiento que usa los tipos semánticos
void procesar_lectura(lectura_sensor_t lectura) {
    printf("Lectura del sensor:\n");
    printf(" - Temperatura: %.1f\n", lectura.temp);
    printf(" - Humedad: %.1f%%\n", lectura.humedad);
    printf(" - Calidad del aire: %d\n", lectura.calidad_aire);
    printf(" - Timestamp: %ld\n", lectura.timestamp);
}
```

```c
struct Sensor {
    int id;
    float temperatura;
};

int main(){
    struct Sensor s1;
    s1.id = 1;
    s1.temperatura = 25.0;

    return 0;
}
```

```c
    // Ejemplo
    typedef struct {
        int id;
        float temperatura;
    } Sensor;

    int main (){
        Sensor s1 = {1, 25.0};
        ...
        return 0;
    }
```

Archivos `.h` y `.c`

Concepto

- Separar declaraciones y definiciones en archivos distintos hace el código más ordenado, reutilizable y escalable.

¿Cómo se organiza?

- .h (header file)
  - #define
  - typedef
  - struct
  - prototipos de funciones
- .c (source file)
  - #include del .h correspondiente
  - implementación de las funciones

`funciones.h`

```h
#ifndef FUNCIONES_H
#define FUNCIONES_H

int sumar(int a, int b);
void imprimirSuma(int a, int b);

#endif
```

funciones.c

```c
#include "funciones.h"
#include <stdio.h>

int sumar(int a, int b) { return a + b; }

void imprimirSuma(int a, int b) {
    printf("La suma es: %d\n", a + b);
}
```

main.c

```c
#include "funciones.h"

int main() {
    imprimirSuma(3, 4);
    return 0;
}
```

⚙️ Directiva #include

Qué hace #include?

- Inserta el contenido de un archivo dentro del código fuente antes de compilar.
- Permite usar módulos externos sin reescribir código.

Dos formas de uso:

#include → busca en las librerías del sistema.

> Ejemplo: **#include** <stdio.h>

#include "archivo.h" → busca en el proyecto del usuario.

> Ejemplo: **#include** "sensor.h"

```c
#include <stdio.h>   // Librerías del sistema
#include "sensor.h"    // Incluimos el módulo

int main() {
    Sensor s1 = {1, 0};              // Inicializamos el sensor con ID y
temperatura en 0
    Sensor *s1ptr = &s1;             // Creamos un puntero al sensor

    leerSensor(s1ptr);              // Simulamos la lectura del sensor

    printf("Sensor %d -> %.1f C\n", s1.id, s1.temperatura);  // Mostramos el
resultado

    return 0;
}
```

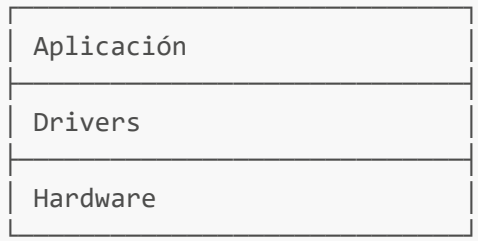
🏗️ Estructura en Capas en Programación

Contenido:

- La programación por capas organiza el código en diferentes niveles con responsabilidades bien definidas.
- Permite modularidad, reutilización y mantenimiento sencillo.
- Cada capa se comunica solo con la capa inmediatamente inferior o superior.

Ejemplo de capa: Una alarma domiciliaria

- **Hardware:** microcontrolador, sensor de movimiento, Sirena (PWM o GPIO).
- **Drivers:** código que abstrae el hardware (ej: driver GPIO, PWM, ADC).
- **Aplicación:** Lógica principal de la alarma:
  - Si un sensor detecta intrusión → activar buzzer y LED rojo
  - Si no hay intrusión → mantener LED verde encendido
  - Posibilidad de armar/desarmar la alarma con un botón



📚 Bibliografía

- *El lenguaje C: Manual de Referencia* — Balestrini & Di Stefano
- *Programación en C para principiantes* — Urrutia & Landa
- *Introducción a la programación en C* — Marzal & Gracia
- *GNU C Language Introduction and Reference Manual* — Richard Stallman

---